

List of Tables

- Table 1: Tools and Technologies Used

List of Figures

- Figure 1: Gantt Chart Representation
- Figure 2: Pipeline Architecture
- Figure 3: GitHub Deployment Triggers
- Figure 4–15: Implementation Screenshots
- Figure 16–23: Results and Monitoring Outputs

Table of Contents

Abstract.....	3
Keywords	4
Role/Task Division	5
Gantt Chart Representation of the Tasks' Duration	6
1.0 Introduction	7
1.1 Background Rationale.....	7
1.2 The Need for a DevOps Solution	7
2. Methodology	8
2.1 Pipeline Architecture And Design.....	8
2.2 Implementation Methodology	9
2.2.1 Development Workflow Design	9
2.2.2 Step-by-Step Implementation Process	9
2.3 Technology Stack	18
3.0 Results And Discussion	19
3.1 GitHub Actions Workflow Result	19
3.1.1 Build and Test Phase Results.....	19
3.1.2 Container Image Build and Push Results	20
3.2 AWS Infrastructure Provisioning Results	21
3.2.1 Amazon ECS Cluster Configuration	21
3.2.2 Application Load Balancer Results	22
3.2.3 Amazon ECR Repository Management.....	23
3.2.4 Amazon RDS PostgreSQL Result	23
3.2.5 Monitoring and Observability with AWS CloudWatch	24
3.4 Challenges and Solutions	25
Conclusion and Future Scope.....	27
Personal Reflection.....	28
Learning Reflection.....	30
References	32
Appendix.....	33

Abstract

This report presents the design and implementation of a cloud-based CI/CD pipeline for multi-component applications, utilising AWS services and modern DevOps practices to address critical challenges in traditional manual deployment processes, such as human error, time consumption, and environmental inconsistencies that hinder software delivery efficiency.

The approach took a cloud-native, containerized strategy using GitHub Actions for continuous integration, Docker for containerization, Terraform for Infrastructure as Code (IaC), and AWS services such as ECS Fargate, ECR, RDS, and Application Load Balancer. The deployment was done using GitFlow branching strategy with automated triggers, with security best practices using AWS Parameter Store for dynamic secrets management and OIDC-based authentication.

Highlight results include major operational gains, which include reducing deployment time from several hours to 2-3 minutes, automated rollback features, better security with containerization and secrets management, and greater scalability with serverless container orchestration. The pipeline fully automated the complete deployment lifecycle from code commit through production deployment, covering infrastructure provisioning, application build, testing, containerization, and deployment verification.

The five-member project team worked well together with specified roles on this project: DevOps engineering and system architecture, documentation and design, development of a rollback system, AWS infrastructure management, and containerization. The key challenges that were resolved included choosing between EC2 and ECS Fargate for compute services, addressing Terraform configuration issues, and managing secure runtime credentials.

This DevOps deployment confirms the viability of cloud-native DevOps methodologies for enabling fast, consistent, and secure software releases and the pipeline automation offers a repeatable template for multi-component applications while laying the groundwork for upcoming additions such as multi-cloud compatibility, sophisticated deployment

techniques, and improved observability. The project illustrates that measured investment in DevOps automation delivers considerable payoff over the long term, reconciling efficiency, security, and maintainability in contemporary software development contexts.

Keywords

DevOps; CI/CD Pipeline; AWS; Docker; Terraform; GitHub Actions; Cloud-Native; Infrastructure as Code; Container Orchestration; Automated Deployment.

Role/Task Division

This project was carried out by a team of five members who had different duties and roles as described below:

- Binawei (HE33207) – DevOps Engineer and System Architect: I developed the pipeline deployment flow, overall system design, and tools and technology selection for the pipeline; created GitHub Actions CI/CD workflows as well as Infrastructure as Code using Terraform for implementation; and integrated AWS services including Amazon Elastic Container Service (ECS), Amazon Elastic Container Registry (ECR), Amazon Relational Database Services (RDS), and Application Load Balancer (ALB).
- Sukhawant (HE33565)—Documentation and System Architecture Designer: Sukhawant focused on the architectural design and documentation, developing detailed system and deployment flow diagrams.
- Daniel (HE33169) – Rollback System Developer: Responsible for developing automated rollback systems. He implemented health monitoring procedures, disaster recovery procedures, and incident response processes to ensure the safety and recovery of deployments.
- Stanley (HE32996)—AWS Infrastructure and Repository Management: Set up an AWS account and IAM user for the pipeline and configure GitHub repository with the AWS IAM user credentials using GitHub secrets.
- Rajbir (HE32864) – Containerisation With Docker: Responsible for containerization by creating the Dockerfiles to create secure and optimized Docker images and containers.

Team collaboration was facilitated using git and gitHub for task management, with daily stand-up meetings to support coordination and progress tracking.

Gantt Chart Representation of the Tasks' Duration

The project was officially started on June 1st, 2025, and lasted up to the 10th of August, 2025, after the initial learning of the required knowledge to carry out the tasks. The corresponding timelines for each project stage and their associated tasks are illustrated in the Gantt charts shown in the figure below.



Figure 1: Gantt Chart representation of the tasks Duration for the DevOps pipeline

1.0 Introduction

1.1 Background Rationale

The modern reality of software development is faced with increased complexity, as applications with many components require frequent deployments to diverse environments. Yet, traditional manual deployment practices have significant shortcomings. These include high potential for human error, where configuration mistakes, missed steps, and environment discrepancies can happen (Humble and Farley, 2010); substantial time consumption, as the deployment of complex applications can take several hours, thus reducing the frequency of deployments; and environment discrepancies, which lead to the notorious "works on my machine" problem, when configurations differ across development, staging, and production environments. In addition, manual rollback processes are time-consuming and error-prone during critical outages, and such practices poorly scale with team growth or the growth of application complexity.

1.2 The Need for a DevOps Solution

DevOps emerged to close the gap between operations and development teams, by emphasising automation, collaboration, and continuous improvement (Kim et al., 2016). Faustino et al. (2021) identified key drivers for the implementation of DevOps solutions:

1. Business Drivers:
 - a. Quick Time-to-Market: Automated pipelines facilitate several daily deployments compared to monthly manual releases
 - b. Better Quality: Automated testing and consistent environments cut production defects by as much as 60%
 - c. Cost Savings: Automation of infrastructure lowers operational overhead and human resource needs
 - d. Competitive Advantage: Quick feature delivery and reliability of the system yield market differentiation
2. Technical Drivers:
 - a. Microservices Architecture for Multi-component Applications Needs Coordinated Deployment Strategies
 - b. Cloud Adoption: Cloud-native applications require Infrastructure as Code practices
 - c. Container Technology: Docker and container orchestration need automated deployment pipelines
 - d. Compliance Requirements: Automated audit trails and repeatable deployments aid in regulatory compliance

2. Methodology

2.1 Pipeline Architecture And Design

The DevOps pipeline implementation follows a cloud-native, containerized approach based on established DevOps principles, which are automation, frequent integration of code changes, incorporating testing at various stages, providing feedback and continuous improvement, as stated by Kim et al. (2016). The architecture adopts a pipeline-as-code using infrastructure-as-code (IaC) methodology, where all deployment processes are version-controlled and automated, ensuring reproducibility and consistency across different environments (Larrucea *et al.*, 2018).

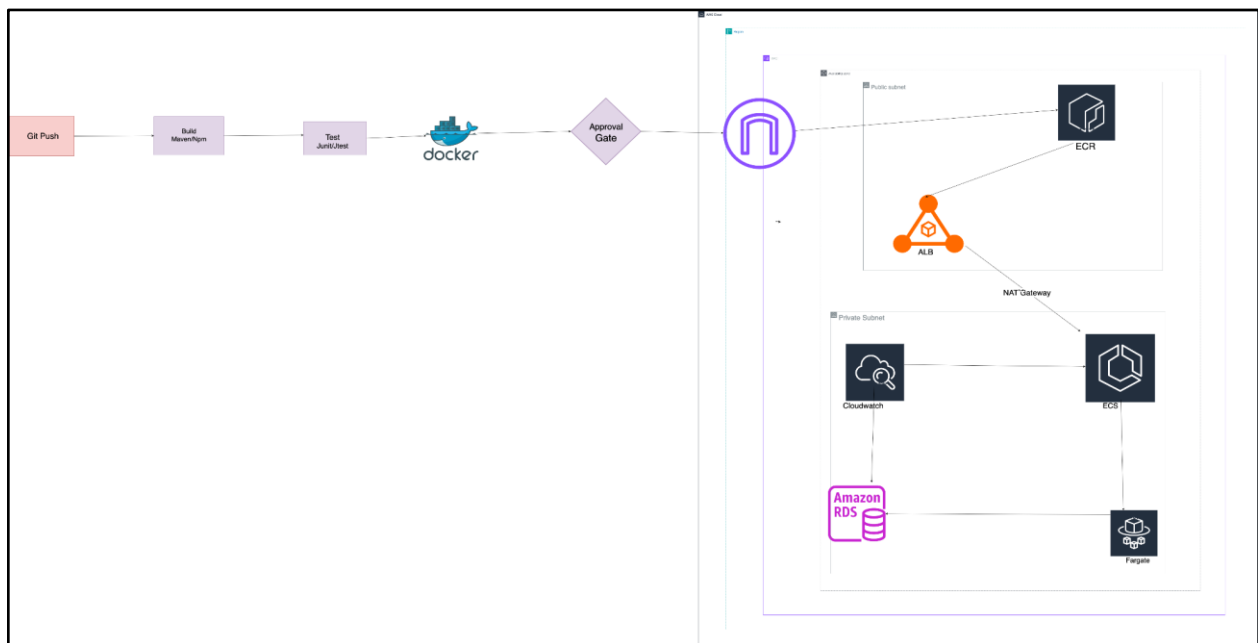


Figure 2: End-to-End Pipeline Architecture from GitHub to AWS

Figure 2 above illustrates the complete data flow from source code commits to production deployments, showing all intermediate stages and AWS services involved.

When code is pushed to GitHub, this automatically triggers a GitHub Actions workflow that checks out the code, installs the necessary environment, performs tests, and builds the application. The application is then packaged into a Docker image, tagged, and pushed to Amazon Elastic Container Registry (ECR). With Terraform, the required AWS infrastructures, like the Virtual Private Cloud (VPC), subnets, and load balancer are created or updated. A new ECS task definition is created, the ECS service is then

updated to use the new image, and a rolling deployment is carried out with health checks to make sure the new containers are running well.

2.2 Implementation Methodology

The implementation of the pipeline follows an iterative approach based on continuous integration and continuous deployment (CI/CD) best practices that were suggested by Jani (2023), which are to start small, automate as much as possible, encourage collaboration, and monitor and improve the system. This pipeline design emphasizes automation, security, and scalability throughout the deployment lifecycle, incorporating feedback loops and quality gates at each stage to ensure reliability and maintainability.

2.2.1 Development Workflow Design

The development workflow implements GitFlow branching strategy with automated triggers on git push, which establishes a structured approach to code integration and deployment (Driessen, 2010). This strategy provides clear separation between development, staging, and production environments while maintaining code quality through automated validation processes. This is shown in the figure below

```
1  name: Build and Deploy RiderApp
2
3  on:
4    push:
5      branches: [main, master, develop, feature/*]
6    workflow_dispatch:
7
```

Figure 3: Trigger configuration for automated deployments

This configuration ensures deployments are triggered automatically on code git push to specified branches, supporting both automated and manual deployment scenarios (Zhu, Bass, and Champlin-Scharff, 2016).

2.2.2 Step-by-Step Implementation Process

The following steps were followed for the implementation of the devops pipeline.

Step 1: Repository Setup and Configuration

The initial setup involves configuring the GitHub repository with the necessary secrets using github secrets and workflow files:

Repository secrets

New repository secret

Name	Last updated
 AWS_ACCESS_KEY_ID	3 weeks ago  
 AWS_SECRET_ACCESS_KEY	3 weeks ago  
 EC2_SSH_KEY	3 weeks ago  

Figure 4: GitHub Secrets Configuration Interface

Figure 4 above shows how AWS credentials are securely stored in GitHub Secrets to enable authentication with AWS services during the deployment process, which follows security best practices for CI/CD pipelines (Chen, 2015).

Step 2: Containerization Implementation

The application to be deployed uses Docker multi-stage builds to optimize image size and security as its containerization process, so the Dockerfile will be added to the project directory as shown below.

```

1 FROM eclipse-temurin:17-jre
2
3 # Install AWS CLI for parameter store access
4 RUN apt-get update && apt-get install -y curl unzip && \
5     curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip" && \
6     unzip awscliv2.zip && \
7     ./aws/install && \
8     rm -rf awscliv2.zip aws && \
9     apt-get clean
10
11 WORKDIR /app
12 COPY target/*.jar app.jar
13
14 # Create startup script that fetches DB credentials
15 COPY docker/start.sh /app/start.sh
16 RUN chmod +x /app/start.sh
17
18 RUN groupadd -r appuser && useradd -r -g appuser appuser
19 RUN chown -R appuser:appuser /app
20 USER appuser
21
22 HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
23     CMD curl -f http://localhost:8080/actuator/health || exit 1
24
25 EXPOSE 8080
26 ENTRYPOINT ["/app/start.sh"]

```

Figure 4: Application Dockerfile content for Dockerisation

In this Dockerfile, the base image is Eclipse Temurin JRE 17, which is a lightweight Java runtime environment, and AWS CLI is installed to allow the container to connect to the AWS environment and to retrieve database credentials and configuration from AWS Parameter Store during runtime. Dynamic configuration fetching is done by the custom startup script (start.sh) prior to starting the application. By creating a non-root user (appuser) to run the application, we improve security by adhering to the principle of least privilege.

Step 3: Infrastructure as Code Implementation

Terraform configurations was used to define AWS infrastructures using Hashicorp language (HCL).

```
# Main Terraform configuration for ECS Fargate deployment
resource "aws_ecs_cluster" "main" {
  name = "${var.project_name}-cluster"

  setting {
    name = "containerInsights"
    value = "enabled"
  }

  tags = {
    Name      = "${var.project_name}-cluster"
    Environment = var.environment
  }
}

resource "aws_ecs_service" "app" {
  name            = "${var.project_name}-service"
  cluster         = aws_ecs_cluster.main.id
  task_definition = aws_ecs_task_definition.app.arn
  desired_count   = var.desired_count
  launch_type     = "FARGATE"

  network_configuration {
    security_groups = [aws_security_group.ecs_tasks.id]
    subnets        = aws_subnet.private[*].id
    assign_public_ip = false
  }

  load_balancer {
    target_group_arn = aws_lb_target_group.app.arn
    container_name   = var.project_name
    container_port    = 8080
  }
}
```

Figure 5: *Terraform configuration for AWS resources*

This Terraform script in Figure 5 illustrates Infrastructure as Code concepts by declaring the AWS resources, such as ECS, in a declarative way. The ECS cluster is set up with Container Insights enabled for improved monitoring features. The ECS service definition in the scripts specifies the state of application deployment, such as utilising the Fargate launch type for serverless container running, network configuration for security isolation, and load balancer integration for traffic distribution. The script utilizes variables to make the configuration reusable across projects and environments

Figure 6 below shows the Terraform output in GitHub Actions.

```
498     + "name" = "riderapp-ecs-tasks-sg"
499   }
500   + vpc_id           = "vpc-041cd465403136311"
501 }
502
503 Plan: 5 to add, 0 to change, 0 to destroy.
504
505 Changes to Outputs:
506 + ecr_repository_url = "964191654598.dkr.ecr.us-east-1.amazonaws.com/riderapp"
507 + ecs_cluster_name   = "riderapp-cluster"
508 + ecs_service_name   = "riderapp-service"
509 + load_balancer_dns  = "riderapp-alb-305274337.us-east-1.elb.amazonaws.com"
510
511 aws_ecs_cluster.main: Creating...
512 aws_security_group.alb: Creating...
513 aws_ecs_task_definition.app: Creating...
514 aws_ecs_task_definition.app: Creation complete after 0s [id=riderapp]
515 aws_security_group.alb: Creation complete after 3s [id=sg-06bdc63c830219044]
516 aws_security_group.ecs_tasks: Creating...
517 aws_security_group.ecs_tasks: Creation complete after 3s [id=sg-011dd6f5c0069e381]
518 aws_ecs_cluster.main: Still creating... [10s elapsed]
519 aws_ecs_cluster.main: Creation complete after 11s [id=arn:aws:ecs:us-east-1:964191654598:cluster/riderapp-cluster]
520 aws_ecs_service.app: Creating...
521 aws_ecs_service.app: Creation complete after 0s [id=arn:aws:ecs:us-east-1:964191654598:service/riderapp-cluster/riderapp-service]
522
523 Apply complete! Resources: 5 added, 0 changed, 0 destroyed.
524
525 Outputs:
526
527 ecr_repository_url = "964191654598.dkr.ecr.us-east-1.amazonaws.com/riderapp"
528 ecs_cluster_name   = "riderapp-cluster"
529 ecs_service_name   = "riderapp-service"
530 load_balancer_dns  = "riderapp-alb-305274337.us-east-1.elb.amazonaws.com"
```

Figure 6: Terraform Output in GitHub Actions

Step 4: Complete CI/CD Pipeline Configuration

The deployment process is then orchestrated by GitHub Actions through a reusable workflow shown in figure 7 below.

```
name: Setup and Deploy Project

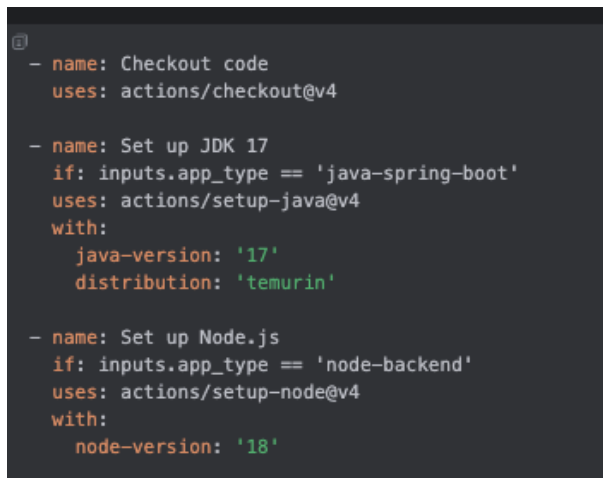
on:
  workflow_call:
    inputs:
      project_name:
        required: true
        type: string
      app_type:
        required: true
        type: string
        description: 'Backend application type (java-spring-boot or node-backend)'
      aws_region:
        required: false
        type: string
        default: "us-east-1"
      enable_database:
        required: false
        type: boolean
        default: false
      database_type:
        required: false
        type: string
        default: "postgres"
      database_instance_class:
        required: false
        type: string
        default: "db.t3.micro"
    secrets:
      AWS_ACCESS_KEY_ID:
        required: true
      AWS_SECRET_ACCESS_KEY:
        required: true

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    environment: production-approval
```

Figure 7: *GitHub Actions workflow*

This workflow configuration establishes a reusable GitHub Actions workflow that can be called by multiple projects with different parameters. The input parameters provide flexibility for different application types (Java Spring Boot or Node.js), AWS regions, and database configurations. The use of workflow_call enables centralised pipeline logic while maintaining project-specific customisations. The production-approval environment ensures that deployments to production require manual approval, implementing a governance control that prevents unauthorized deployments

Step 4.1: Environmental Setup and Code Checkout



```
- name: Checkout code
  uses: actions/checkout@v4

- name: Set up JDK 17
  if: inputs.app_type == 'java-spring-boot'
  uses: actions/setup-java@v4
  with:
    java-version: '17'
    distribution: 'temurin'

- name: Set up Node.js
  if: inputs.app_type == 'node-backend'
  uses: actions/setup-node@v4
  with:
    node-version: '18'
```

Figure 8: stage involving setup and code checkout

These initial steps establish the foundation for the deployment pipeline by preparing the execution environment. The checkout action retrieves the source code from the repository, making it available to subsequent steps. The conditional runtime setup ensures that the appropriate development environment is configured based on the application type like Java 17 with the Temurin distribution for Spring Boot applications or Node.js 18 for Node.js backends.

Step 4.2: Application build and Testing

```
- name: Run tests (Java)
  if: inputs.app_type == 'java-spring-boot'
  run: ./mvnw clean test

- name: Build application (Java)
  if: inputs.app_type == 'java-spring-boot'
  run: ./mvnw clean package -DskipTests

- name: Install dependencies (Node)
  if: inputs.app_type == 'node-backend'
  run: npm ci

- name: Run tests (Node)
  if: inputs.app_type == 'node-backend'
  run: npm test -- --coverage --watchAll=false
```

Figure 9: Application build and Testing stage

The Figure 9 above shows the build and test phase, which implements continuous integration principles by ensuring code quality before deployment. For Java applications, Maven executes the complete test suite followed by application packaging into a deployable JAR file. The clean command ensures a fresh build environment, while -DskipTests in the package step avoids redundant test execution. For Node.js applications, npm ci provides faster, reliable dependency installation from the lock file, followed by comprehensive test execution with coverage reporting.

Step 4.3: AWS Authentication and ECR Integration

```
- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials@v4
  with:
    aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
    aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
    aws-region: ${ inputs.aws_region }

- name: Login to Amazon ECR
  id: login-ecr
  uses: aws-actions/amazon-ecr-login@v2
```

Figure 10: AWS authentication and ECR integration

Figure 10 shows the AWS authentication phase, which establishes secure connectivity between GitHub Actions and AWS services. The credential configuration step uses GitHub Secrets to securely store and access AWS access keys, following security best practices by avoiding hardcoded credentials in the workflow. The ECR login step obtains a temporary authentication token specifically for the Elastic Container Registry,

enabling the pipeline to push Docker images. The ID: login-ecr assignment allows subsequent steps to reference the registry URL output from this authentication process.

Step 4.4: Container Image Build and Push

```
- name: Build and push Docker image
  env:
    ECR_REGISTRY: ${ steps.login-ecr.outputs.registry }
    IMAGE_TAG: ${ github.sha }
  run: |
    # Create ECR repository if it doesn't exist
    aws ecr describe-repositories --repository-names ${ inputs.project_name } || aws ecr create-repository --repository-name ${ inputs.project_name }

    # Build Docker image with Git commit SHA as tag
    docker build -t $ECR_REGISTRY/${ inputs.project_name }:$IMAGE_TAG .

    # Create 'latest' tag for convenience
    docker tag $ECR_REGISTRY/${ inputs.project_name }:$IMAGE_TAG $ECR_REGISTRY/${ inputs.project_name }:latest

    # Push both tagged versions to ECR
    docker push $ECR_REGISTRY/${ inputs.project_name }:$IMAGE_TAG
    docker push $ECR_REGISTRY/${ inputs.project_name }:latest
```

Figure 11: Docker image build and push to ECR

The above figure 11 illustrates the containerization stage that takes the application built and converts it into a Docker image and stores it in Amazon ECR. It starts by making sure that the ECR repository is present, and if not, it creates one. The Docker build process makes use of the current directory context (which has the Dockerfile) to build an image tagged with the Git commit SHA to uniquely identify each build.

Step 4.5: Infrastructure State Management

The next stage is the setup the provisioning of AWS infrastructures using terraform as shown in the figure 12 below

```
- name: Setup Terraform
  uses: hashicorp/setup-terraform@v3
  with:
    terraform_version: 1.5.0

- name: Check if ECS services already exist
  run: |
    EXISTING_CLUSTER=$(aws ecs describe-clusters \
      --clusters ${ inputs.project_name }-cluster \
      --query 'clusters[0].status' \
      --output text 2>/dev/null || echo "MISSING")

    echo "Existing cluster status: $EXISTING_CLUSTER"

    if [ "$EXISTING_CLUSTER" = "ACTIVE" ]; then
      echo "CLUSTER_EXISTS=true" >> $GITHUB_ENV
      echo "Found existing ECS cluster, will update service"
    else
      echo "CLUSTER_EXISTS=false" >> $GITHUB_ENV
      echo "No existing cluster, Terraform will create new infrastructure"
    fi
```

Figure 12: infrastructure state management

Figure 12 is the infrastructure setup stage that sets up the environment for AWS resource management. Terraform is installed with a version to provide reproducible infrastructure operations across pipeline runs. It also checks for the cluster's existence to ascertain if this is a first-time deployment or an update to an existing infrastructure.

Step 4.6: Infrastructure Provisioning with Terraform

This step uses the terraform commands to deploy the required infrastructures as shown in the figure 13 below

```
- name: Deploy infrastructure with Terraform
  env:
    TF_VAR_project_name: ${inputs.project_name}
    TF_VAR_app_type: ${inputs.app_type}
    TF_VAR_aws_region: ${inputs.aws_region}
    TF_VAR_enable_database: ${inputs.enable_database}
    TF_VAR_database_type: ${inputs.database_type}
    TF_VAR_database_instance_class: ${inputs.database_instance_class}
    TF_VAR_image_uri: ${steps.login-ecr.outputs.registry}/${inputs.project_name}:${github.sha}
  run: |
    cd devops/terraform
    terraform init
    terraform plan
    terraform apply -auto-approve

    echo "LOAD_BALANCER_DNS=$(terraform output -raw load_balancer_dns)" >> $GITHUB_ENV
    echo "ECR_REPOSITORY_URL=$(terraform output -raw ecr_repository_url)" >> $GITHUB_ENV
    echo "ECS_CLUSTER_NAME=$(terraform output -raw ecs_cluster_name)" >> $GITHUB_ENV
    echo "ECS_SERVICE_NAME=$(terraform output -raw ecs_service_name)" >> $GITHUB_ENV
```

Figure 13: deploying infrastructures with terraform

The Terraform apply command deploys Infrastructure as Code by provisioning and configuring all necessary AWS resources automatically, the environment variables provide the workflow inputs and calculated values (such as the Docker image URI) to terraform configurations. Terraform's three-phase process of initialization, planning, and application provides a reliable way to deploy infrastructure. Terraform init installs necessary providers, terraform plan checks and previews changes, and the terraform apply applies the infrastructure changes.

Step 4.7: ECS Fargate Deployment Process

```
- name: Deploy to ECS Fargate
  env:
    ECR_REGISTRY: ${steps.login-ecr.outputs.registry}
    IMAGE_TAG: ${github.sha}
  run: |
    TASK_DEFINITION=$(aws ecs describe-task-definition --task-definition ${inputs.project_name} --query 'taskDefinition')
    echo "$TASK_DEFINITION" > /tmp/task-def.json

    jq --arg IMAGE "$ECR_REGISTRY/${inputs.project_name}:${IMAGE_TAG}" '
    .containerDefinitions[0].image = $IMAGE |
    del(.taskDefinitionArn, .revision, .status, .requiresAttributes, .placementConstraints, .compatibilities, .registeredAt, .registeredBy, .tags)
    ' /tmp/task-def.json > /tmp/new-task-def.json

    NEW_TASK_DEF_ARN=$(aws ecs register-task-definition --cli-input-json file:///tmp/new-task-def.json --query 'taskDefinition.taskDefinitionArn' --)

    aws ecs update-service \
      --cluster ${env.ECS_CLUSTER_NAME} \
      --service ${env.ECS_SERVICE_NAME} \
      --task-definition $NEW_TASK_DEF_ARN

    aws ecs wait services-stable \
      --cluster ${env.ECS_CLUSTER_NAME} \
      --services ${env.ECS_SERVICE_NAME}
```


Figure 14: ECS Fargate Deployment stage

This stage implements a rolling update strategy by creating a new task definition with the updated container image and then updating the service to use this new definition. The process gets the current task definition, modifies it to point to the new Docker image, and removes AWS-managed metadata that might hinder registration. The jq tool is used for JSON manipulation to change the container image pointer while keeping other configuration settings in place. The service section updates ECS to progressively replace the running containers with the new ones, and the wait command ensures the deployment is successful before proceeding.

Step 4.8: Deployment Verification and Output

```
- name: Output deployment info
  run: |
    echo "🎉 Deployment completed successfully!"
    echo "🌐 Application URL: http://${ env.LOAD_BALANCER_DNS }"
    echo "🐳 Docker Image: ${ env.ECR_REPOSITORY_URL }:${ env.github.sha }"
    echo "🚀 ECS Cluster: ${ env.ECS_CLUSTER_NAME }"
    echo "🏠 ECS Service: ${ env.ECS_SERVICE_NAME }"
```

Figure 15: deployment output

The final step provides deployment confirmation information, which shows successful deployment, and provides the application URL for immediate testing, documenting the specific Docker image version deployed, and listing AWS resource names for operational reference.

2.3 Technology Stack

The following technologies and CI/CD tools, which were identified by Jani (2023) in Table 1 were used to build and integrate with the devops CI/CD pipeline (Shirinkin, 2017; Bheri and Vummenthala, 2019; AWS, 2025)

Table 1: tools and technologies

Technology	Description	Justification
Github Actions	A (CI/CD) platform that helps automate the build, test, and deployment pipelines.	Native GitHub integration and free of charge
Docker	It is a platform used for developing, shipping, and running	Industry standard for containerization.

	applications. It helps you separate your applications from your infrastructure so you can deliver software quickly.	
Terraform	An infrastructure-as-code tool that lets you build, change, and version cloud infrastructure safely and efficiently.	Declarative IaC, multi-cloud support
Amazon Elastic Container Registry (ECR)	A fully managed Docker container registry that makes it easy to store, share, and deploy container images	Integrates seamlessly with AWS services such as ECS and provides strong security features
Amazon Elastic Container Service (ECS)	A fully managed container orchestration service that helps you easily deploy, manage, and scale containerized applications	Provides native AWS integration for running Docker containers at scale without managing Kubernetes.
AWS Fargate	A serverless compute engine for containers, primarily used with Amazon ECS	Removes the need to provision or manage servers
Application Load Balancer (ALB)	Distributes incoming traffic across multiple targets	Supports path-based and host-based routing.
Amazon Relational Database Service (RDS)	A web service that makes it easier to set up, operate, and scale a relational database in the AWS Cloud.	Provides a fully managed PostgreSQL database

3.0 Results And Discussion

3.1 GitHub Actions Workflow Result

The implemented pipeline demonstrates successful automation across all deployment stages. The GitHub Actions interface provides comprehensive visibility into the deployment process.

build-and-deploy / build-and-deploy

succeeded 2 days ago in 3m 34s

Q

Search logs

↺

⚙

>

✔ Checkout code

1s

>

✔ Set up JDK 17

0s

>

✔ Run tests

14s

>

✔ Build application

8s

>

✔ Configure AWS credentials

0s

>

✔ Login to Amazon ECR

1s

>

✔ Build and push Docker image

46s

>

✔ Setup Terraform

1s

>

✔ Check if ECS services already exist

1s

>

✔ Deploy infrastructure with Terraform

43s

>

✔ Deploy to ECS Fargate

1m 34s

>

✔ Output deployment info

0s

>

✔ Post Login to Amazon ECR

0s

>

✔ Post Configure AWS credentials

1s

>

✔ Post Set up JDK 17

0s

>

✔ Post Checkout code

0s

>

✔ Complete job

0s

Figure 16: *GitHub Actions Workflow Overview*

The Figure 16 above is the complete execution workflow of all steps, their status, and time of completion, which shows a successful deployment process in GitHub Actions.

3.1.1 Build and Test Phase Results

The automated build process in the pipeline completes within 1-2 minutes, including compilation, testing, and packaging.

```
Build application 8s

1 ▶ Run ./mvnw clean package -DskipTests
7 [INFO] Scanning for projects...
8 [INFO]
9 [INFO] -----< com.tev:RiderApp >-----
10 [INFO] Building RiderApp 0.0.1-SNAPSHOT
11 [INFO] from pom.xml
12 [INFO] -----[ jar ]-----
13 Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-jar-plugin/3.4.2/maven-jar-plugin-3.4.2.pom
14 Progress (1): 1.4/7.7 kB
15 Progress (1): 2.8/7.7 kB
16 Progress (1): 4.1/7.7 kB
17 Progress (1): 5.5/7.7 kB
18 Progress (1): 6.9/7.7 kB
19 Progress (1): 7.7 kB
20
21 Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-jar-plugin/3.4.2/maven-jar-plugin-3.4.2.pom (7.7 kB at 40 kB/s)
22 Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/42/maven-plugins-42.pom
23 Progress (1): 1.4/7.7 kB
24 Progress (1): 2.8/7.7 kB
25 Progress (1): 4.1/7.7 kB
26 Progress (1): 5.5/7.7 kB
27 Progress (1): 6.9/7.7 kB
28 Progress (1): 7.7 kB
29
30 Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-plugins/42/maven-plugins-42.pom (7.7 kB at 699 kB/s)
31 Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-jar-plugin/3.4.2/maven-jar-plugin-3.4.2.jar
1806 Progress (1): 458/752 kB
1807 Progress (1): 475/752 kB
1808 Progress (1): 491/752 kB
1809 Progress (1): 507/752 kB
1810 Progress (1): 524/752 kB
1811 Progress (1): 540/752 kB
1812 Progress (1): 557/752 kB
1813 Progress (1): 573/752 kB
1814 Progress (1): 589/752 kB
1815 Progress (1): 606/752 kB
1816 Progress (1): 622/752 kB
1817 Progress (1): 639/752 kB
1818 Progress (1): 655/752 kB
1819 Progress (1): 671/752 kB
1820 Progress (1): 688/752 kB
1821 Progress (1): 704/752 kB
1822 Progress (1): 720/752 kB
1823 Progress (1): 737/752 kB
1824 Progress (1): 752 kB
1825
1826 Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-collections4/4.4/commons-collections4-4.4.jar (752 kB at 6.7 MB/s)
1827 [INFO] Replacing main artifact /home/runner/work/RiderApp/RiderApp/target/RiderApp-0.0.1-SNAPSHOT.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
1828 [INFO] The original artifact has been renamed to /home/runner/work/RiderApp/RiderApp/target/RiderApp-0.0.1-SNAPSHOT.jar.original
1829 [INFO] -----
1830 [INFO] BUILD SUCCESS
1831 [INFO] -----
1832 [INFO] Total time: 5.780 s
1833 [INFO] Finished at: 2025-08-15T16:15:16Z
1834 [INFO] -----
```

Figure 17: Build Phase Execution Details

Figure 17 shows logs from the Maven build process that shows successful compilation and package execution.

3.2.2 Container Image Build and Push Results

The Docker image creation and push operations to Amazon ECR were successful, which demonstrates efficient containerization of the pipeline. This is shown in Figure 3.3 below.

```
Build and push Docker image
1 ▶ Run # Create ECR repository if it doesn't exist
20 {
21   "repositories": [
22     {
23       "repositoryArn": "arn:aws:ecr:us-east-1:964191654598:repository/riderapp",
24       "registryId": "964191654598",
25       "repositoryName": "riderapp",
26       "repositoryUri": "964191654598.dkr.ecr.us-east-1.amazonaws.com/riderapp",
27       "createdAt": "2025-08-05T10:28:45.669000+00:00",
28       "imageTagMutability": "MUTABLE",
29       "imageScanningConfiguration": {
30         "scanOnPush": false
31       },
32       "encryptionConfiguration": {
33         "encryptionType": "AES256"
34       }
35     }
36   ]
37 }
38 #0 building with "default" instance using docker driver
39
40 #1 [internal] load default definition from Dockerfile
41 #1 transferring dockerfile: 799B done
42 #1 DONE 0.0s
43
44 #2 [auth] library/eclipse-temurin:pull token for registry-1.docker.io
45 #2 DONE 0.0s
46
47 #3 [internal] load metadata for docker.io/library/eclipse-temurin:17-jre
48 #3 DONE 0.4s

0989 1e2cf0aa63bf: Pushed
0990 bfa8f4c09a53: Layer already exists
0991 935d9ad6b2f6: Layer already exists
0992 17aae5bfaa7c: Layer already exists
0993 249c5eafc821: Layer already exists
0994 273c6b37e353: Pushed
0995 5b10fd088491: Pushed
0996 104ec5c9f017: Pushed
0997 cd9666b1462e: Pushed
0998 15b2cabb717c: Pushed
0999 23e43b2b314c20363241c741111cc5c208c36fc0: digest: sha256:1ac0c1c48809b5654514ccdd4aa9ebc3e277e8f4e910a1e99ca0647b138a5fd14 size: 2032
0980 The push refers to repository [964191654598.dkr.ecr.us-east-1.amazonaws.com/riderapp]
0981 5b10fd088491: Preparing
0982 273c6b37e353: Preparing
0983 a85d4fabe1ef: Preparing
0984 64a922460e99: Preparing
0985 104ec5c9f017: Preparing
0986 1e2cf0aa63bf: Preparing
0987 15b2cabb717c: Preparing
0988 bfa8f4c09a53: Preparing
0989 935d9ad6b2f6: Preparing
0990 17aae5bfaa7c: Preparing
0991 249c5eafc821: Preparing
0992 cd9666b1462e: Waiting
0993 1e2cf0aa63bf: Waiting
0994 15b2cabb717c: Waiting
0995 bfa8f4c09a53: Waiting
0996 935d9ad6b2f6: Waiting
0997 17aae5bfaa7c: Waiting
0998 249c5eafc821: Waiting
0999 cd9666b1462e: Waiting
0980 a85d4fabe1ef: Layer already exists

53
54 #5 [1/8] FROM docker.io/library/eclipse-temurin:17-jre@sha256:357917f5448855c2d88b362059dc089c26dcab8a8b0885446b5a46c8b9e49
55 #5 resolve docker.io/library/eclipse-temurin:17-jre@sha256:357917f5448855c2d88b362059dc089c26dcab8a8b0885446b5a46c8b9e49 done
56 #5 sha256:357917f5448855c2d88b362059dc089c26dcab8a8b0885446b5a46c8b9e49 8.48kB / 8.48kB done
57 #5 sha256:1d8ffca381da1462fe080a41dbed6981831e3886af8847e79e093d0f7a62 1.94kB / 1.94kB done
58 #5 sha256:826e44053714642229aaaf7f88808eade224677ac795389e48e978c082 5.98kB / 5.98kB done
59 #5 sha256:b7146094f2604c2e8881749670e588aa7a8f64ca4acdf26c45e4bde7e4e 5.24MB / 29.72MB 0.1s
60 #5 sha256:7685e75e5ebc595322712a4779810488bcae7f6a4706aaf73dc690d739df 2.18MB / 16.97MB 0.1s
61 #5 sha256:b7146094f2604c2e8881749670e588aa7a8f64ca4acdf26c45e4bde7e4e 11.53MB / 16.97MB 0.1s
62 #5 sha256:b7146094f2604c2e8881749670e588aa7a8f64ca4acdf26c45e4bde7e4e 11.53MB / 16.97MB 0.2s
63 #5 sha256:7685e75e5ebc595322712a4779810488bcae7f6a4706aaf73dc690d739df 14.69MB / 16.97MB 0.2s
64 #5 sha256:b7146094f2604c2e8881749670e588aa7a8f64ca4acdf26c45e4bde7e4e 14.69MB / 16.97MB 0.2s
65 #5 ...
66
67 #6 [internal] load build context
68 #6 transferring context: 79.70MB 0.3s done
69 #6 DONE 0.3s

71 #5 [1/8] FROM docker.io/library/eclipse-temurin:17-jre@sha256:357917f5448855c2d88b362059dc089c26dcab8a8b0885446b5a46c8b9e49
72 #5 sha256:b7146094f2604c2e8881749670e588aa7a8f64ca4acdf26c45e4bde7e4e 24.12MB / 29.72MB 0.4s
73 #5 sha256:7685e75e5ebc595322712a4779810488bcae7f6a4706aaf73dc690d739df 16.97MB / 16.97MB 0.2s done
74 #5 sha256:b7146094f2604c2e8881749670e588aa7a8f64ca4acdf26c45e4bde7e4e 31.40MB / 46.99MB 0.4s
75 #5 sha256:826e44053714642229aaaf7f88808eade224677ac795389e48e978c082 159B / 159B 0.3s done
76 #5 sha256:c2097665d6c20640081c0a0aef37366e047029372a8dc08ba0a89045 2.28kB / 2.28kB 0.3s done
77 #5 sha256:b7146094f2604c2e8881749670e588aa7a8f64ca4acdf26c45e4bde7e4e 28.13MB / 29.72MB 0.5s
78 #5 sha256:b7146094f2604c2e8881749670e588aa7a8f64ca4acdf26c45e4bde7e4e 38.89MB / 46.99MB 0.5s
79 #5 sha256:b7146094f2604c2e8881749670e588aa7a8f64ca4acdf26c45e4bde7e4e 29.72MB / 29.72MB 0.5s done
80 #5 sha256:b7146094f2604c2e8881749670e588aa7a8f64ca4acdf26c45e4bde7e4e 44.83MB / 46.99MB 0.6s
81 #5 extracting sha256:b7146094f2604c2e8881749670e588aa7a8f64ca4acdf26c45e4bde7e4e 0.1s
82 #5 sha256:b7146094f2604c2e8881749670e588aa7a8f64ca4acdf26c45e4bde7e4e 46.99MB / 46.99MB 0.7s done
83 #5 extracting sha256:b7146094f2604c2e8881749670e588aa7a8f64ca4acdf26c45e4bde7e4e 0.9s done
84 #5 extracting sha256:7685e75e5ebc595322712a4779810488bcae7f6a4706aaf73dc690d739df
85 #5 extracting sha256:7685e75e5ebc595322712a4779810488bcae7f6a4706aaf73dc690d739df 0.6s done
```

Figure 18: Docker Build and ECR Push Logs

3.2 AWS Infrastructure Provisioning Results

The Terraform automation, through its configuration, successfully provisioned all required AWS resources as shown below:

3.2.1 Amazon ECS Cluster Configuration

Figure 19 below presents the provisioned ECS cluster overview, showcasing active services, tasks, and capacity providers.

The ECS cluster shows the service status, which shows active with the desired task count maintained, task definitions, and scaling configuration, which scales automatically based on the CPU/memory utilization.

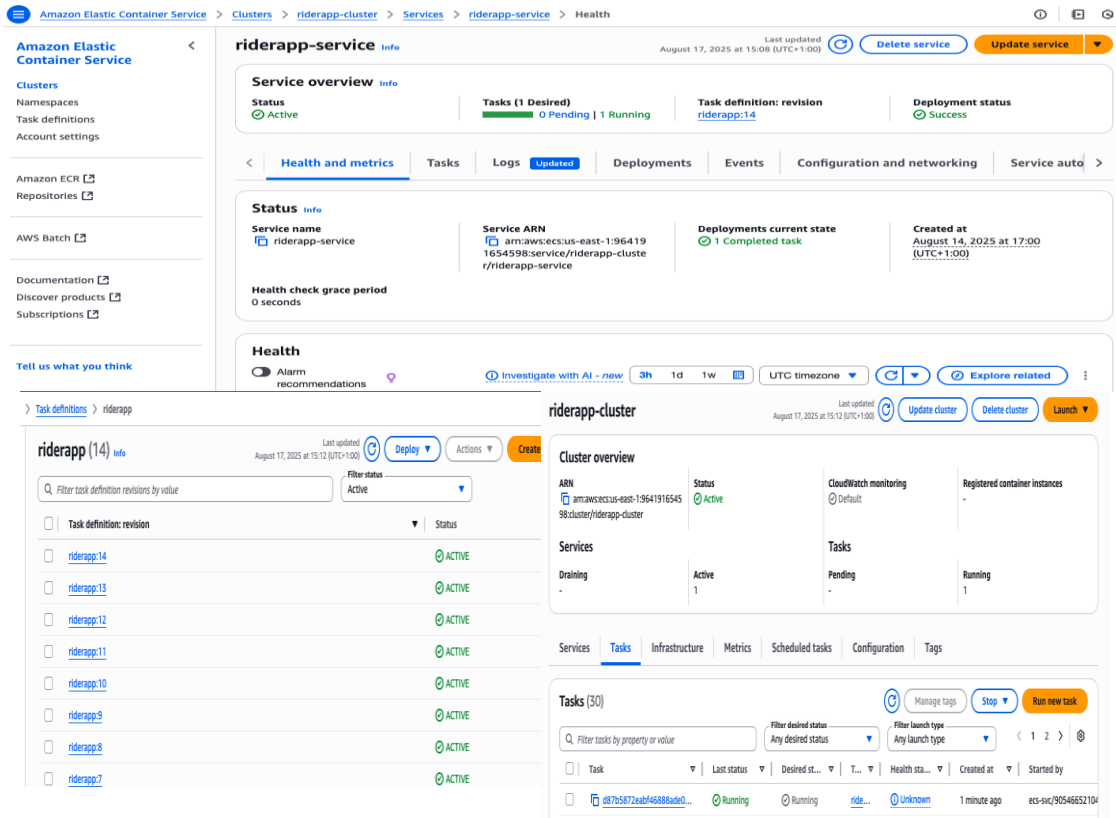


Figure 19: AWS ECS Cluster Dashboard

3.3.2 Application Load Balancer Results

The figure below shows the provisioned the application load balancer on the AWS console

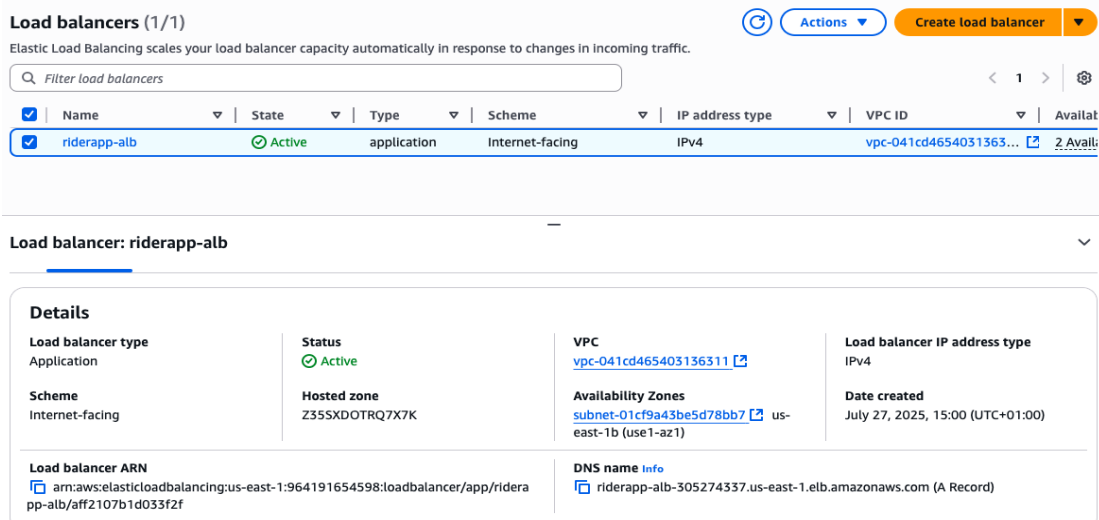


Figure 20: Alb configuration output

The AWS application load balancer is for routing traffic to the provisioned services from the internet.

3.3.3 Amazon ECR Repository Management

The screenshot shows the Amazon ECR console interface. On the left, there's a navigation menu with 'Private registry' selected. The main area shows the 'Images (30)' page for the 'riderapp' repository. A table lists several image versions, including 'latest', 'fb9204d9bef79da66a72e1e17a533ea2949dc1d6', 'ec24edd803360143e5f69cd9168aa720bc80f083', and '3926cd5e64a28e543269460c452e939e2fad9b80'. Each row shows the image tag, artifact type (Image), pushed at time, size (MB), image URI, digest, and last time. Below the table, the 'Details' section for the 'latest' image is expanded, showing its URI, digest, and general information.

Image tag	Artifact type	Pushed at	Size (MB)	Image URI	Digest	Last time
latest, 23e43b28314c28363241c741111cc5c208c36fc0	Image	August 15, 2025, 17:16:03 (UTC+01)	331.72	Copy URI	sha256:1ac0c1c488095b...	Aug 10:3
fb9204d9bef79da66a72e1e17a533ea2949dc1d6	Image	August 15, 2025, 17:11:48 (UTC+01)	331.72	Copy URI	sha256:459d1b86b3d430...	-
ec24edd803360143e5f69cd9168aa720bc80f083	Image	August 14, 2025, 16:59:55 (UTC+01)	331.67	Copy URI	sha256:f4339335aeb5a1...	Aug 16:4
3926cd5e64a28e543269460c452e939e2fad9b80	Image	August 14, 2025, 16:55:01 (UTC+01)	331.67	Copy URI	sha256:1c6bf31c76f44c9...	-

Image Details

Image tags
latest, 23e43b28314c28363241c741111cc5c208c36fc0

URI
964191654598.dkr.ecr.us-east-1.amazonaws.com/riderapp:latest

Digest
sha256:1ac0c1c488095b654514ccd4aa9ebc3e277e8f4e918a1e99ca0647b138a5fd14

General information

Artifact type	Repository	Pushed at
Image	riderapp	August 15, 2025, 17:16:03 (UTC+01)

Last recorded pull time	Size (MB)
August 17, 2025, 10:39:52 (UTC+01)	331.72

Figure 21: ECR Repository with Image Versions

Figure 21 above shows the ECR repository interface where the Docker images are stored, showing their version and image SHA, and security scanning for automated vulnerability assessment.

3.3.4 Amazon RDS PostgreSQL Result

Figure 22 below shows the RDS PostgreSQL instance configuration with backup and monitoring settings, which has a metric of connection pool utilization of 60-70%, and automated daily backups with 7-day retention.

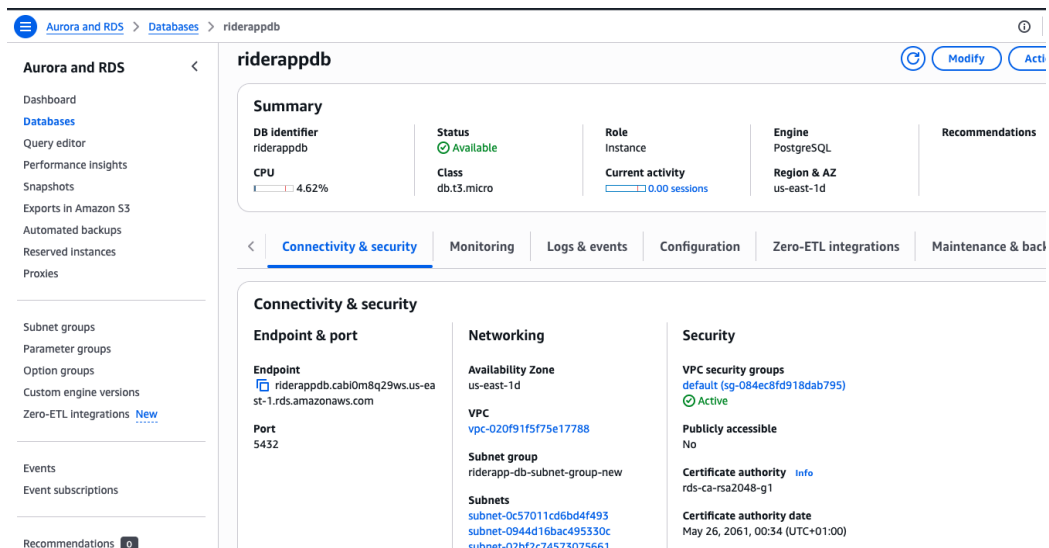


Figure 22: RDS Database Instance Details

3.3.5 Monitoring and Observability with AWS CloudWatch

All provisioned or updated AWS services are monitored and observed on AWS CloudWatch, as shown in Figure 23 below, which displays the monitoring dashboard showing application and infrastructure metrics, log streams, and log events.

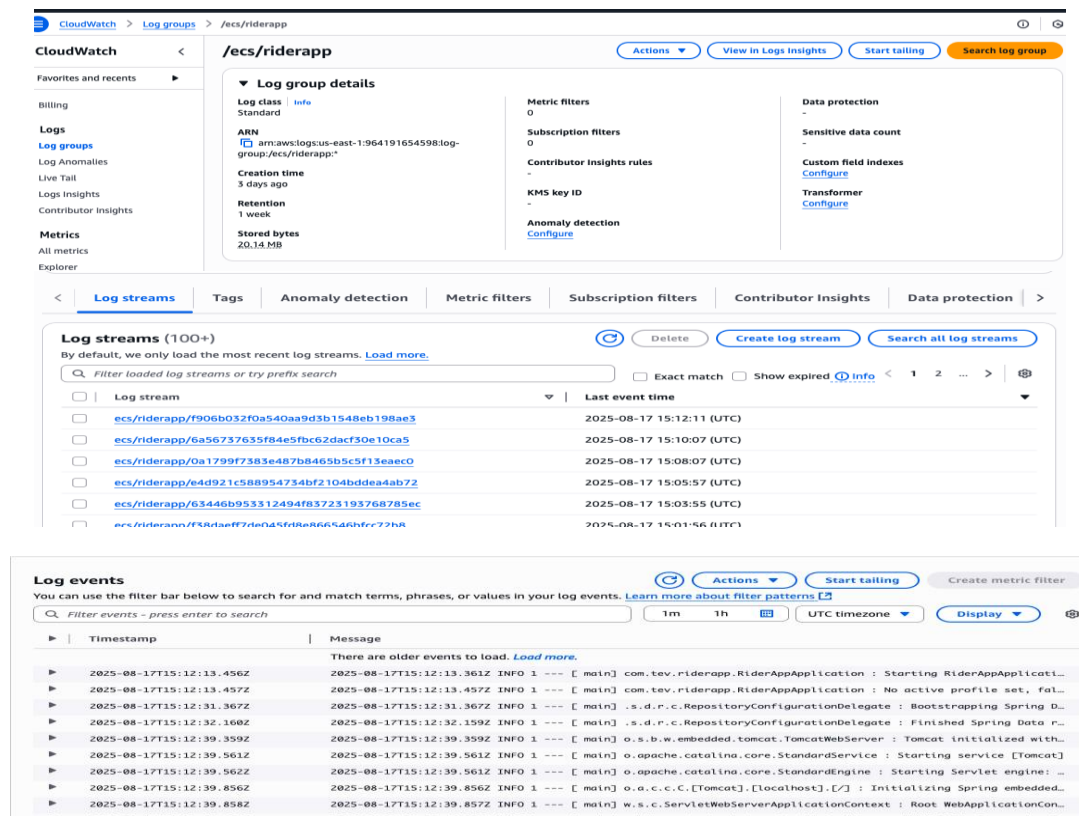


Figure 23: CloudWatch Dashboard Overview

One notable outcomes of the devops pipeline was the reduction in deployment time from several hours to less than three minutes which accelerates release cycles and enables teams to respond faster to customer needs and urgent security vulnerabilities, and the integration of automated rollback mechanisms further enhanced reliability by ensuring quick recovery from failed releases, thereby reducing Mean Time to Recovery (MTTR) which shows how automation can change the software delivery lifecycle, and relieving engineers from repetitive manual tasks while reducing the risk of human error.

The findings also aligns with established DevOps principles as identified by Maroukian and Gulliver (2020) who argued that automation promotes consistency and reduces configuration errors, and this project confirmed that by eliminating manual steps in deployment also Fitzgerald and Stol (2017) emphasised the role of continuous software engineering in supporting rapid iteration, a goal clearly achieved through the pipeline's automation of build, test, and deployment phases, and this project demonstrated the value of Infrastructure as Code (IaC), with Terraform ensuring reproducibility and maintainability across environments (Scarfone et al., 2018).

Some identified strength of this pipeline implementation includes the drastic improvement in speed and efficiency illustrates how DevOps practices can enable rapid value delivery, also reliability was improved through automated rollback and health monitoring, ensuring higher service availability, scalability was achieved using ECS Fargate, which dynamically manages container workloads without requiring manual intervention and as well security was strengthened by integrating OIDC authentication and AWS Parameter Store, thereby avoiding the risks associated with hardcoded credentials, which all shows that automation can balance speed, security, and scalability--factors that are often seen as competing priorities in traditional software delivery models.

Some identified limitations are vendor lock-in, as the pipeline uses AWS services such as ECS, RDS, ECR, and CloudWatch, so migrating to another provider would require substantial reconfiguration, which reduces flexibility. The deployment strategy also remains limited to rolling updates. While effective, this approach does not offer the fine-grained control of advanced techniques such as blue-green or canary deployments, which are widely recognised as safer approaches for high-stakes production systems. Similarly, while monitoring through CloudWatch provided useful system-level insights, deeper observability could be achieved by incorporating distributed tracing tools like OpenTelemetry.

Some trade-offs that were made during the design of the pipeline includes the choice of Amazon ECS Fargate over AWS EC2 due to its serverless nature and reduced management burden and as ECS was free, GitHub Actions was selected for CI/CD because of its integration with GitHub and flexibility. Similarly, Terraform was preferred over AWS CloudFormation for its multi-cloud portability, reflecting a conscious choice to prioritise long-term adaptability over short-term vendor-specific optimisation (Morris, 2016). These trade-offs illustrate how architectural decisions in DevOps must balance simplicity, cost, and scalability.

3.4 Challenges and Solutions

The following are challenges that I faced during the development of the DevOps pipeline.

1. **Compute Services Selection between EC2 and ECS Fargate:** Choosing between EC2 and ECS as the compute service to run the containerized applications was one challenge I encountered, where I evaluated Amazon ECS which itself is free as a container orchestration service, which integrates with Fargate to provide a serverless model with automatic scaling, built-in security,

and seamless AWS integration, and EC2 offered full infrastructure control and cost efficiency for steady workloads but introduced significant management overhead, including patching, scaling, and monitoring responsibilities. Balancing these trade-offs was a critical architectural decision that directly impacted efficiency, maintainability, and long-term scalability of the deployment (Pahl et al., 2016).

2. **Terraform Syntax and Configuration Errors:** There were also errors relating to HashiCorp Configuration Language (HCL) syntax, which made the deployment pipeline to fail, as configuration files initially contained errors like the use of semicolons (;), which are valid in traditional programming languages but are not in HCL's declarative syntax. This caused initialization and planning failures, emphasizing the need to shift the mindset from imperative coding practices to declarative infrastructure definitions. I had to learn declarative infrastructure management, then resolve the issue by restructuring the configuration according to HCL best practices, ensuring clearer, more maintainable, and error-free Terraform code (Morris, 2016).
3. **Dynamic Configuration and Secrets Retrieval:** Another challenge was how to implement a secure database credential management without hardcoding sensitive information into container images, which requires retrieving database credentials at runtime, avoids embedding secrets in images, as well as enabling credential rotation without requiring application restarts. This was addressed by using a dual-layer secret management approach which was implemented using AWS Parameter Store, which provided secure, centralized storage and seamless integration with containerized applications.

Conclusion and Future Scope

In conclusion, this project illustrated the implementation of a cloud-native CI/CD pipeline for a multi-component application using AWS ECS Fargate, AWS ECR, Docker, Terraform, and GitHub Actions, and illustrates how an automated pipeline can enhance deployment efficiency, reliability, and scalability, cutting down deployment time from multiple hours to less than 2-3 minutes. Containerization using Docker and Amazon ECR integration provided environmental consistency across various environments like development and production, while dual-layer secret management improved security by fetching credentials dynamically without baking sensitive data into images.

Some challenges that were encountered during the development of the pipeline include the selection of the right compute service, involving weighing the operational ease and serverless capabilities of ECS Fargate versus the complete control and cost predictability of EC2 instances. Terraform syntax and declarative infrastructure management presented an initial learning curve, which was resolved by refactoring configurations based on HCL best practices. Secure secrets and runtime configuration fetching were solved using AWS Parameter Store, enabling centralized, runtime-managed credentials supporting multi-environment deployments. These resolutions underscore the value of embracing cloud-native principles and DevOps automation to effectively cope with complexity.

In addition, the pipeline also provides more rapid, dependable deployments, which facilitated regular feature releases, minimized the risk of human error, and enhanced developer productivity. And the secure, scalable, and automated deployment process also provides a reproducible pattern for other multi-component apps, in addition to forming the basis of additional future improvements like multi-cloud support, sophisticated deployment techniques, and enhanced observability procedures. In general, this implementation validates that investment in cloud-native DevOps procedures yields great long-term rewards, equilibrating efficiency, security, and maintainability for contemporary software development.

Personal Reflection

Throughout this project, I was the architect and developer of the DevOps deployment section of the pipeline, which required me to engage deeply in multiple dimensions of the project lifecycle. I was fully involved in the technical execution through to strategic decision-making, problem-solving, and project management. My contribution to the design and development system's architecture and selection of the cloud technologies that are used in the pipeline, particularly the use of AWS ECS Fargate over EC2 deployments, reflects careful consideration of operational complexity, cost implications, and long-term maintainability (Bass et al., 2015).

The architectural component I selected set this project apart from conventional approaches. For instance, I implemented a secret management system that separated CI/CD pipeline credentials from runtime application secrets, addressing a common security challenge in containerized deployments while maintaining operational flexibility (Kim et al., 2016). I also developed a systematic technology selection and integration strategy, which evaluates alternatives within the AWS ecosystem, considering long-term operational impact, team skill development, and alignment with organizational objectives (Fowler and Lewis, 2014). Connecting GitHub Actions with AWS services and using OIDC-based authentication needed me to focus on security, best practices, and how to organise workflows, allowing me to use my cloud security knowledge effectively.

Problem-solving and innovation were central to my role throughout the project. I encountered numerous technical challenges, including Terraform configuration errors and complex AWS IAM permission issues, which required persistent investigation and creative solutions (Morris, 2016). And through parameterized GitHub Actions workflows, I demonstrated modularity, reusability, and maintainability in infrastructure automation (Fowler and Lewis, 2014), developed an integrated automated testing within the CI/CD pipeline, and implemented monitoring and observability on AWS CloudWatch (Kim et al., 2016). These efforts reflect my commitment to operational excellence and recognize deployment as the beginning of the application lifecycle.

This project helped me to get hands-on experience in cloud computing, particularly with AWS services such as ECS, ECR, RDS, and Application Load Balancer, and advanced from theoretical understanding to practical mastery (Armbrust et al., 2010). I also gained experience in declarative infrastructure management with Terraform, which fundamentally shifted my approach to system design and deployment (Kief, 2017). Beyond that, I developed stronger capabilities in communication, documentation, critical

thinking, and strategic analysis by explaining complex technical concepts to diverse audiences and documenting problem-solving approaches enhanced my effectiveness in collaborative environments and contributed to knowledge transfer in an organization.

Finally, this project provided me with enriching experiences in resilience, adaptability, and confidence-building as I persevered through successive configuration errors and challenging system integrations that initially triggered imposter syndrome; however, the successful deployment of the DevOps pipeline solution reaffirmed my professional identity and growth (Morris, 2016). I also learned to balance depth of knowledge with project momentum, prioritizing critical activities, and maintaining quality when time-limited has strengthened my project management and decision-making capabilities. The project not only enabled technical contribution and operations improvement but also enabled significant personal and professional development, setting me up strongly for future leadership and innovation in DevOps and cloud architecture.

Learning Reflection

Through this project, I learned how to create environments that are versionable, reproducible, and testable using Terraform Infrastructure as Code using the Hashicorp Configuration Language (HCL), as well as working with AWS services, including ECS Fargate, RDS, and Application Load Balancer, which provided deep insight into cloud service integration, orchestration, and dependency management (Armbrust et al., 2010; Kief, 2017), and the complexity of IAM permission management highlighted the importance of systematic approaches to cloud security and increased my understanding of the principle of least privilege (Scarfone et al., 2018).

I also developed practical knowledge of container technologies and orchestration practices as I gained skills in containerization with Docker, Docker image optimization, implementing non-root security measures, provisioning infrastructures using Terraform, and container orchestration with ECS Fargate (Pahl et al., 2016), which clarified the differences between container platforms, the merits of serverless orchestration, and best practices for secure and sustainable deployment.

Also, I understand more of the DevOps culture and methods, as I gained the skills to balance automation with required human intervention, create approval gateways for production deployments, and develop rollback scenarios to manage risk efficiently (Kim et al., 2016). From these activities, I gained a deep insight into organizational and cultural aspects of DevOps transformation, recognizing that technical implementation is inextricably tied to process improvement, governance, and change management. In addition, the creation of reusable, parameterized workflows highlighted the importance of modularity, maintainability, and scalability of CI/CD system design (Fowler & Lewis, 2014).

Beyond the technical knowledge, I also gained project management, communication, and analytical problem-solving skills, and risk and contingency planning were critical habits, as did deconstructing intricate projects into manageable pieces while still having end-to-end visibility (Larrucea et al., 2018).

I gained hands-on experience in system design, including consideration of component interactions, emergent system properties, scalability, and maintainability (Bass et al., 2015). The assessment of computing and architectural options, trade-off balancing between security, cost, performance, and operational complexity, and comprehension of the long-term consequences of initial decisions sharpened analytical capabilities and strategic thinking. Exposure to new trends in serverless computing, GitOps, and

sophisticated monitoring practices reinforced sensitivity to industry progression and future directions for cloud-native architectures (Armbrust et al., 2010; Kim et al., 2016). Also, continually being confronted with challenges requires persistence, adaptability, and resilience, thus underlining the value of a growth mindset. Recognition that proficiency is developed through practice, reflection, and iterative processes instilled a commitment to lifelong learning and professional development (Larrucea et al., 2018).

References

AWS 2025 <https://docs.aws.amazon.com/>

Bheri, S. and Vummenthala, S., 2019. An Introduction to the DevOps Tool Related Challenges.

Faustino, J., Adriano, D., Amaro, R., Pereira, R. and da Silva, M.M., 2022. DevOps benefits: A systematic literature review. *Software: Practice and Experience*, 52(9), pp.1905-1926. <https://doi.org/10.1002/spe.3096>

Gupta, A. (2024) *DevOps Monitoring and Logging: Tools and Techniques*. International Journal of Advanced and Innovative Research, 10(1), pp. 713-714.

Humble, J. and Farley, D., 2010. *Continuous delivery: reliable software releases through build, test, and deployment automation*. Pearson Education.

Kim, G., Humble, J., Debois, P., Willis, J. and Forsgren, N., 2021. *The DevOps handbook: How to create world-class agility, reliability, & security in technology organizations*. It Revolution.

Larrucea, X., Santamaria, I., Colomo-Palacios, R. and Ebert, C., 2018. Microservices. *IEEE Software*, 35(3), pp.96–100. <https://doi.org/10.1109/MS.2018.2141030>.

Maroukian, K. and Gulliver, S.R., (2020). Leading DevOps practice and principle adoption. *arXiv preprint arXiv:2008.10515*.

Morris, K., 2016. *Infrastructure as code: managing servers in the cloud*. " O'Reilly Media, Inc."

Pahl, C., Brogi, A., Soldani, J., and Jamshidi, P. (2016). Cloud container technologies: A state-of-the-art review. *IEEE Transactions on Cloud Computing*, 7(3), 677-692.

Shirinkin, K., 2017. *Getting Started with Terraform*. Packt Publishing Ltd.

Scarfone, K., Souppaya, M., Cody, A., and Orebaugh, A. (2018). Technical guide to information security testing and assessment. NIST Special Publication 800-115.

Sether, A. (2016) *Cloud computing benefits*. Social Science Electronic Publishing (SSRN), pp. 5-11. Available from: <https://ssrn.com/abstract=2781593> or <https://dx.doi.org/10.2139/ssrn.2781593> [Accessed 4 June 2025].

Thokala, V.S. (2021) *Utilizing Docker Containers for Reproducible Builds and Scalable Web Application Deployments*. International Journal of Current Engineering and Technology, 11(6), pp.661-668.

- Ugwueze, V.U. and Chukwunweike, J.N. (2024) *Continuous integration and deployment strategies for streamlined DevOps in software engineering and application delivery*. International Journal of Computer Applications Technology and Research, 14(1), pp.1, 13-15.
- Xu, T. and Zhou, Y. (2015) *Systems approaches to tackling configuration errors: A survey*. ACM Computing Surveys (CSUR), 47(4), p.12.
- Zhu, L., Bass, L. and Champlin-Scharff, G., 2016. DevOps and its practices. *IEEE Software*, 33(3), pp.32–34. doi:10.1109/MS.2016.81.

Appendix

Github Repository: <https://github.com/Binawei/DevopsPipeline>